

BUSINESS REQUIREMENTS DOCUMENT

Finance Data Processing and Access Control Backend

1. Executive Summary

This Business Requirements Document (BRD) formally defines the functional and non-functional requirements for the Finance Data Processing and Access Control Backend system. The system is being designed as a backend internship technical assessment for Rajkaran Yadav.

The proposed system is a RESTful backend service powering a multi-role finance dashboard. It manages financial records, enforces role-based access control, and provides aggregated analytics endpoints to support data-driven decisions by finance teams.

This BRD captures the business context, stakeholder needs, functional requirements, data model, access control rules, API scope, validation expectations, and optional enhancements — serving as the blueprint for system design and implementation.

2. Business Context & Problem Statement

2.1 Business Context

Organizations managing financial data across multiple user roles — such as viewers, analysts, and administrators — require a structured backend capable of maintaining data integrity while selectively exposing information based on user permissions. Without such a system, data mismanagement, unauthorized modifications, and inaccurate financial summaries become significant operational risks.

2.2 Problem Statement

There is a need for a centralized backend service that:

- Stores and manages financial records (transactions, entries) in a structured and queryable manner.
- Enforces role-based access control (RBAC) at the API layer, ensuring each user type can only perform permitted operations.
- Provides dashboard-ready summary APIs returning aggregated financial data for consumption by a frontend interface.
- Handles validation and errors gracefully to ensure reliability and clear developer feedback.

2.3 Business Objectives

- Build a reliable, well-structured backend for a finance dashboard with clear separation of concerns.
- Demonstrate API design, data modelling, business logic, and access control competency.
- Deliver a maintainable codebase with documented assumptions and trade-offs.
- Provide clean and consistent error handling that reflects production-level thinking.

3. Project Scope

3.1 In Scope

- User management: creation, role assignment, and status management (active/inactive).
- Role-based access control with at least three roles: Viewer, Analyst, and Admin.
- Financial records CRUD: create, read, update, delete, and filter operations.
- Dashboard summary APIs: total income, total expenses, net balance, category-wise totals, and monthly trends.
- Input validation, error handling, and meaningful HTTP status codes.
- Data persistence using a database or structured in-memory store.

3.2 Out of Scope

- Frontend / UI development — this is purely a backend system.
- Multi-tenancy or organization-level data isolation beyond individual user roles.
- Production-grade deployment, scaling, or SLA commitments.
- Real financial integrations (bank APIs, payment gateways, etc.).
- Advanced reporting formats (PDF, CSV export) — unless added as optional enhancements.

4. Stakeholders

Stakeholder	Role	Interest / Expectation
Rajkaran Yadav	Applicant / Developer	Delivers a well-structured, functional backend demonstrating engineering judgment.
Evaluating Team	Hiring / Assessment Panel	Evaluates backend design, logic, code quality, and documentation clarity.
End Users (System)	Viewer / Analyst / Admin	Interact with the finance dashboard per their role permissions.

5. Functional Requirements

5.1 User and Role Management

The system shall provide APIs or service logic to manage users and their roles within the finance dashboard.

ID	Requirement Description	Priority	Applicable Role
UR-01	The system shall allow creation of new users with a name, email, password, and assigned role.	High	Admin
UR-02	The system shall support assignment or update of roles (Viewer, Analyst, Admin) for existing users.	High	Admin
UR-03	The system shall allow toggling of user status between active and inactive.	Medium	Admin
UR-04	The system shall list all registered users with their roles and statuses.	Medium	Admin
UR-05	Inactive users shall be denied access to all protected endpoints upon authentication.	High	System
UR-06	User records shall include a unique identifier, creation timestamp, and last updated timestamp.	High	System

5.2 Financial Records Management

The system shall support full lifecycle management of financial entries (income, expenses, and other transaction types).

ID	Requirement Description	Priority	Applicable Role
FR-01	The system shall allow creation of financial records with fields: amount, type (income/expense), category, date, and optional notes.	High	Admin
FR-02	The system shall allow retrieval of all financial records, with support for filtering by date range, type, and category.	High	Admin, Analyst, Viewer
FR-03	The system shall allow updating of existing financial records including all editable fields.	High	Admin
FR-04	The system shall allow deletion (or soft-deletion) of financial records.	High	Admin
FR-05	Each financial record shall carry a unique ID, owner reference, and timestamps (created/updated).	High	System
FR-06	Filtering shall support combinations: e.g., all expenses in a date range for a given category.	Medium	Admin, Analyst, Viewer
FR-07	Records shall support soft delete — deleted records are flagged as inactive but not permanently removed.	Low	Admin

5.3 Dashboard Summary APIs

The system shall expose aggregated summary endpoints to support a finance dashboard frontend.

ID	Requirement Description	Priority	Applicable Role
DS-01	The system shall return total income, total expenses, and net balance as a summary object.	High	Admin, Analyst, Viewer
DS-02	The system shall return category-wise totals for both income and expense types.	High	Admin, Analyst
DS-03	The system shall return recent financial activity (e.g., last N records, configurable via query param).	Medium	Admin, Analyst, Viewer
DS-04	The system shall return monthly or weekly trend data — aggregated totals grouped by time period.	Medium	Admin, Analyst
DS-05	All summary APIs shall accept optional date range filters to scope the aggregation window.	Medium	Admin, Analyst

6. Access Control Requirements

Role-based access control (RBAC) shall be enforced at the API layer through middleware, guards, or policy checks. The following matrix defines allowed operations per role.

Operation	Viewer	Analyst	Admin
View financial records	✓ Allowed	✓ Allowed	✓ Allowed
Filter financial records	✓ Allowed	✓ Allowed	✓ Allowed
View dashboard summary	✓ Allowed	✓ Allowed	✓ Allowed
Access trend / category analytics	✗ Denied	✓ Allowed	✓ Allowed
Create financial records	✗ Denied	✗ Denied	✓ Allowed
Update financial records	✗ Denied	✗ Denied	✓ Allowed
Delete financial records	✗ Denied	✗ Denied	✓ Allowed
Create / manage users	✗ Denied	✗ Denied	✓ Allowed
Assign roles	✗ Denied	✗ Denied	✓ Allowed
Toggle user active/inactive	✗ Denied	✗ Denied	✓ Allowed
View user list	✗ Denied	✓ Allowed	✓ Allowed

6.1 Access Control Implementation Notes

- Access control shall be implemented as reusable middleware or decorator components.
- Unauthorized access attempts shall return HTTP 403 Forbidden with a descriptive message.
- Unauthenticated requests to protected endpoints shall return HTTP 401 Unauthorized.

- Role enforcement shall occur before business logic execution.

7. Data Model Requirements

7.1 User Entity

Field	Type	Constraints	Description
id	UUID / Integer	Primary Key	Unique user identifier
name	String	Required, max 255	Full name of the user
email	String	Required, Unique	Email address used for login
password_hash	String	Required	Hashed password (bcrypt or equivalent)
role	Enum	Required	One of: VIEWER, ANALYST, ADMIN
is_active	Boolean	Default: true	Whether user account is active
created_at	Timestamp	Auto-set	Record creation timestamp
updated_at	Timestamp	Auto-updated	Last modification timestamp

7.2 Financial Record Entity

Field	Type	Constraints	Description
id	UUID / Integer	Primary Key	Unique record identifier
amount	Decimal	Required, > 0	Monetary value of the record
type	Enum	Required	One of: INCOME, EXPENSE
category	String	Required, max 100	Category label (e.g., Salary, Rent, Travel)
date	Date	Required	Date of the financial event
notes	Text	Optional	Additional description or remarks
created_by	FK → User.id	Required	Reference to the admin who created the record
is_deleted	Boolean	Default: false	Soft delete flag
created_at	Timestamp	Auto-set	Record creation timestamp
updated_at	Timestamp	Auto-updated	Last modification timestamp

8. API Endpoint Specifications

8.1 Authentication

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user (Admin seeding or open)
POST	/api/auth/login	No	Authenticate user; returns JWT or session token
POST	/api/auth/logout	Yes	Invalidate current session/token

8.2 User Management

Method	Endpoint	Role Required	Description
GET	/api/users	Admin	List all users with roles and statuses
POST	/api/users	Admin	Create a new user and assign a role
PATCH	/api/users/:id/role	Admin	Update a user's assigned role
PATCH	/api/users/:id/status	Admin	Toggle user active/inactive status
DELETE	/api/users/:id	Admin	Remove a user from the system

8.3 Financial Records

Method	Endpoint	Role Required	Description
GET	/api/records	All Roles	List records; supports filter params (type, category, date_from, date_to)
GET	/api/records/:id	All Roles	Fetch a single financial record by ID
POST	/api/records	Admin	Create a new financial record
PUT	/api/records/:id	Admin	Full update of an existing record
PATCH	/api/records/:id	Admin	Partial update (e.g., update only notes)
DELETE	/api/records/:id	Admin	Soft-delete a financial record

8.4 Dashboard Summary

Method	Endpoint	Role Required	Description
GET	/api/dashboard/summary	All Roles	Returns total income, expenses, and net balance
GET	/api/dashboard/categories	Admin, Analyst	Category-wise income and expense breakdown
GET	/api/dashboard/recent	All Roles	Recent transactions (configurable via ?limit=N)

Method	Endpoint	Role Required	Description
GET	/api/dashboard/trends	Admin, Analyst	Monthly/weekly aggregated trend data

9. Non-Functional Requirements

9.1 Validation & Error Handling

- All incoming request payloads shall be validated for type, format, and required fields before processing.
- Validation errors shall return HTTP 422 Unprocessable Entity with a field-level error detail array.
- Business rule violations (e.g., duplicate user email) shall return HTTP 400 Bad Request.
- Resource not found scenarios shall return HTTP 404 Not Found with a descriptive message.
- Internal server errors shall return HTTP 500 with a sanitized message (no stack trace in production).
- All error responses shall follow a consistent JSON structure: { error: string, details?: any }.

9.2 Security

- Passwords shall be hashed using bcrypt or equivalent before storage.
- Authentication tokens (JWT or session) shall be validated on every protected request.
- Role checks shall not be bypassable via query parameter manipulation or header injection.
- Sensitive fields (e.g., password_hash) shall never be returned in API responses.

9.3 Performance (Assessment Context)

- Response times should be reasonable for a local development setup; no hard SLAs required.
- Dashboard aggregation queries should avoid N+1 patterns — use database-level aggregation where possible.

9.4 Maintainability

- Code shall follow a clear layered architecture: routes/controllers, services/business logic, data access/models.
- Shared logic (auth guards, validators, error handlers) shall be extracted to reusable modules.
- Environment-specific configuration (database URL, JWT secret) shall be managed via environment variables.

10. Optional Enhancements

The following features are not mandatory but demonstrate additional engineering depth if implemented:

Enhancement	Description	Value Added
JWT Authentication	Stateless token-based auth with expiry and refresh logic.	High
Pagination	Cursor or offset-based pagination on list endpoints with page/limit params.	High
Search Support	Full-text or fuzzy search on record notes or category fields.	Medium
Soft Delete	Records flagged as deleted rather than permanently removed; restorable by Admin.	Medium
Rate Limiting	Per-user or per-IP request throttling to prevent API abuse.	Medium
Unit / Integration Tests	Test coverage for service logic, access control enforcement, and edge cases.	High
API Documentation	Swagger/OpenAPI spec or equivalent accessible documentation.	High

11. Assumptions & Constraints

11.1 Assumptions

- Mock or local authentication is acceptable; full OAuth/SSO is not required.
- A single currency is assumed for all financial records (multi-currency support is out of scope).
- There is a single system-wide category list; no per-user category customization.
- An Admin account will be seeded during initial setup for testing purposes.
- The system operates in a single timezone; no timezone conversion logic is required.
- Financial records are assumed to be in the past or present — future-dated records are allowed but not validated specially.

11.2 Constraints

- The system is built for a local development and assessment environment, not for production deployment.
- No external financial or banking APIs are to be integrated.
- The data persistence layer must be documented; in-memory stores are acceptable if noted.
- The submission is a one-time final submission; no iterative review cycles are expected.

12. Evaluation Criteria Alignment

This BRD has been structured to align with the evaluation criteria defined by the assessment. The table below maps each criterion to the corresponding BRD section and planned implementation approach.

Evaluation Criterion	BRD Section	Planned Approach
Backend Design	Section 8	Layered architecture: routes, services, models; clean separation of concerns
Logical Thinking	Sections 5, 6	Role enforcement via middleware; business rules enforced in service layer
Functionality	Section 5, 8	All CRUD + filter + summary endpoints implemented and tested manually
Code Quality	Section 9.4	Consistent naming, modular structure, no dead code, clear comments
Database & Data Modelling	Section 7	Normalized schema with FK references, indexes on frequently filtered fields
Validation & Reliability	Section 9.1	Validation at controller layer; consistent error shape across all endpoints
Documentation	README + BRD	Detailed README with setup, assumptions, API reference, and design notes
Additional Thoughtfulness	Section 10	Optional enhancements like JWT, pagination, Swagger, soft delete

13. Document Sign-Off

This Business Requirements Document represents a complete and accurate capture of the requirements for the Finance Data Processing and Access Control Backend internship assignment.

Prepared and submitted by:

Rajkaran Yadav

Name: Rajkaran Yadav

Role: Backend Developer Intern Applicant

End of Document — Finance Data Processing and Access Control Backend BRD v1.0